

SMC PLIC — Functional Specification

Contents

- SMC PLIC — Functional Specification
 - Contents
 - 1. Purpose & scope
 - 2. Conformance & references
 - 3. Feature summary
 - 4. Configuration parameters
 - 4.1 CCI parameters (primary interface)
 - 4.2 `plic_cfg` struct (backward-compatibility defaults)
 - 5. Block diagram & port list
 - 5.1 Block diagram
 - 5.2 Ports
 - 5.3 Context mapping (default `num_contexts = 8`)
 - 6. Theory of operation
 - 6.1 Operating principle
 - 6.2 Architectural concepts
 - 6.3 Pipeline view
 - 6.4 Source-level lifecycle
 - 6.5 Per-context arbitration
 - 6.6 Threshold semantics — worked example
 - 6.7 Hardware ↔ software interaction
 - 6.8 Edge cases
 - 6.9 Steady-state characteristics
 - 7. Register map
 - 7.1 Region summary
 - 7.2 PRIORITY — `0x000000 + 4·src`
 - 7.3 PENDING — `0x001000 + 4·word`
 - 7.4 ENABLE — `0x002000 + 0x80·ctx + 4·word`
 - 7.5 THRESHOLD — `0x200000 + 0x1000·ctx + 0x000`
 - 7.6 CLAIM/COMPLETE — `0x200000 + 0x1000·ctx + 0x004`
 - 7.7 Address arithmetic helpers
 - 8. Functional behaviour
 - 8.1 Source latching (gateway)
 - 8.2 Per-context arbitration

- 8.3 Claim / complete handshake
- 8.4 Source 0 reservation
- 8.5 Output-update model
- 9. Reset behaviour
- 10. Bus interface (TLM-2.0 / AXI4-Lite)
 - 10.1 Socket type
 - 10.2 Transaction contract
 - 10.3 Timing annotation
 - 10.4 Optional `smc_axi_extension`
 - 10.5 `transport_dbg`
- 11. Error handling
- 12. Programming model
 - 12.1 Initialisation sequence (firmware)
 - 12.2 ISR template
 - 12.3 Worked example
- 13. Compliance matrix
 - 13.1 Documented deviations
 - 13.2 Unmodelled features
- 14. Revision history
- 15. Glossary

SMC PLIC — Functional Specification

Document: `01_PLIC_Specification.md` **Module:** `smc::plic` (SystemC/TLM-2.0 Loosely-Timed model)

Spec base: RISC-V Platform-Level Interrupt Controller (PLIC) v1.0 **Status:** Frozen for SMC bring-up; tracks `plic.rdl` and `riscv_plic0.c` **Companion docs:**

- `02_PLIC_LowLevel_Design.md` — internal SystemC implementation
 - `03_PLIC_Test_Plan.md` — verification strategy & test list
-

Contents

1. [Purpose & scope](#)
 2. [Conformance & references](#)
 3. [Feature summary](#)
 4. [Configuration parameters](#)
 5. [Block diagram & port list](#)
 6. [Theory of operation](#)
 7. [Register map](#)
 8. [Functional behaviour](#)
 9. [Reset behaviour](#)
 10. [Bus interface \(TLM-2.0 / AXI4-Lite\)](#)
 11. [Error handling](#)
 12. [Programming model](#)
 13. [Compliance matrix](#)
 14. [Revision history](#)
 15. [Glossary](#)
-

1. Purpose & scope

The SMC PLIC is the central **Platform-Level Interrupt Controller** for the SMC chiplet. It aggregates up to 336 device-level interrupt sources into 8 per-context outputs (4 cores × {M-mode, S-mode}) that drive the Rocket cores' `MEIP` / `SEIP` lines.

This specification defines the **externally-observable behaviour** of the SMC PLIC: configuration, register map, interrupt-flow semantics, reset, and bus contract. It is the contract that:

- Firmware (`fw/smc/common/drivers/riscv_plic0.c`) programs against,
- The RTL (`OCAH1CORECluster_TLPLIC.sv`) implements at gate level, and
- The SystemC model (`peripherals/plic/`) implements at transaction level.

Internal implementation choices (process topology, data structures, single-driver scheme, etc.) are documented in `02_PLIC_LowLevel_Design.md`.

2. Conformance & references

Source	Authority
RISC-V PLIC Specification v1.0	Protocol & arbitration semantics
<code>hw/smc/smc_cpu/data/registers/rdl/plic.rdl</code>	Ground-truth register map
<code>hw/smc/smc_cpu/chipyard_generated_files/1core/OCAH1CORECluster_TLPLIC.sv</code>	Functional reference RTL
<code>fw/smc/common/drivers/riscv_plic0.c</code>	Firmware driver (must run unmodified)
<code>01_SMC_Architecture.md §5</code>	SMC-specific sizing parameters
<code>02_SMC_IP_LowLevel_Design.md §4</code>	TLM-2.0 interface conventions

The model is **bit-compatible** with the RDL register layout and protocol- compatible with the RISC-V PLIC v1.0 specification. All deviations are listed explicitly in §13.

3. Feature summary

Feature	Value / behaviour
Standard	RISC-V PLIC v1.0
Sources	336 (default; configurable 1..1023)
Contexts	8 (default; configurable ≥ 1)
Priority levels	8 (3-bit field; 0 = disabled, 7 = highest)
Threshold levels	8 (3-bit field; 0 = pass-all, 7 = mask-all)
Tie-break	Lowest source ID wins
Source type	Active-high, level-sensitive
Pending semantics	Set on rising edge of source line
Claim	Atomic read of <code>CLAIM/COMPLETE</code> register
Complete	Write of source ID to <code>CLAIM/COMPLETE</code> register
Re-arm	On <code>complete()</code> if the source line is still high
Bus interface	TLM-2.0 LT target socket (AXI4-Lite, 32-bit)
Window size	4 MB (<code>0x40_0000</code>)
DMI	Not granted (claim has read side effects)
Reset polarity	Active-low, synchronous

4. Configuration parameters

4.1 CCI parameters (primary interface)

The model exposes three OSCI CCI (`cci_configuration`) parameters. These are the **primary** configuration interface; tools, test benches, and platform integrators should use the CCI broker to set them.

CCI parameter name	Type	Mutability	Default	Range / unit	Notes
<code>num_sources</code>	<code>cci_param<unsigned></code> (<code>CCI_IMMUTABLE_PARAM</code>)	Immutable (locked after elaboration)	336	1 .. 1023	Source IDs are 1-based; ID 0 is reserved. Maps to <code>PRIORITY[337]</code> in <code>plic.rdl</code> and <code>PLIC0_MAX_INTERRUPTS</code> in the firmware driver.
<code>num_contexts</code>	<code>cci_param<unsigned></code> (<code>CCI_IMMUTABLE_PARAM</code>)	Immutable (locked after elaboration)	8	≥ 1	One context per <code>{hart, privilege-mode}</code> pair. SMC default: 4 cores $\times$ {M-mode, S-mode}.
<code>access_delay_ns</code>	<code>cci_param<double></code>	Mutable (can be changed between transactions)	2.0	ns	Annotated delay added to <code>sc_time& delay</code> in <code>b_transport</code> . Approximates AXI4-Lite bus latency.

Hierarchical param names follow the standard CCI convention: `<broker_prefix>.<sc_module_name>.<param_name>`, e.g.:

```
// Override before the module is constructed (preset):
broker.set_preset_cci_value("top.plic.num_sources",
                           cci::cci_value::from_json("64"));
broker.set_preset_cci_value("top.plic.num_contexts",
                           cci::cci_value::from_json("2"));

// Change access delay at any point during simulation (mutable):
auto h = broker.get_param_handle("top.plic.access_delay_ns");
h.set_cci_value(cci::cci_value(5.0));
```

At construction time the PLIC logs a `SC_REPORT_INFO` message showing each parameter's resolved value and whether it came from a CCI preset (`[preset]`) or the constructor default (`[default]`).

4.2 `plic_cfg` struct (backward-compatibility defaults)

The constructor still accepts an optional `plic_cfg cfg` argument. Its `num_sources` and `num_contexts` fields supply the **default values** for the corresponding CCI params. A CCI broker preset set *before* construction takes priority over these defaults.

```
smc::plic dut("plic"); // all CCI defaults (336 / 8 / 2 ns)
smc::plic dut("plic", { .num_sources = 64,
                       .num_contexts = 2 }); // defaults from cfg; preset can still win
```

`plic_cfg` continues to own all fixed address-map constants (unchanged from the RDL); these are not CCI params because they must never vary at run-time.

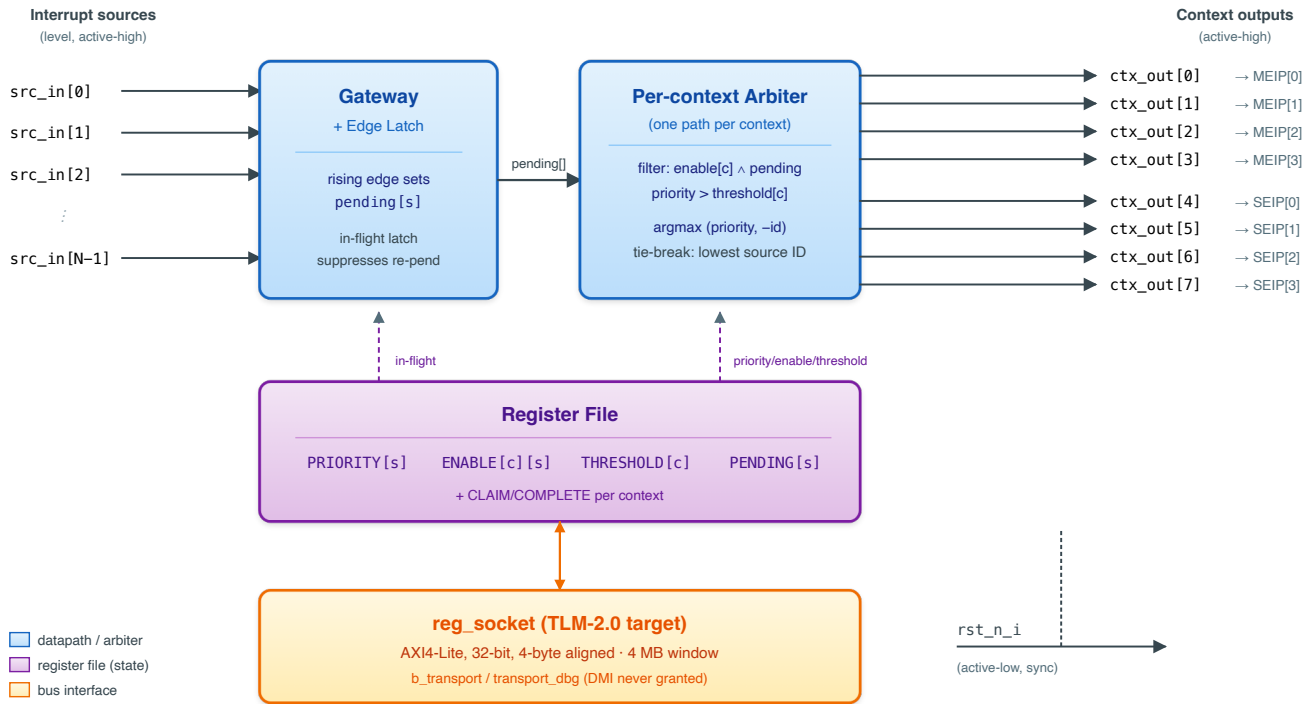
Fixed address-map constants (from `plic.rdl`):

Constant	Value	Description
<code>PRIORITY_BASE</code>	<code>0x000000</code>	Base of <code>PRIORITY[src]</code> registers
<code>PENDING_BASE</code>	<code>0x001000</code>	Base of <code>PENDING[word]</code> registers
<code>ENABLE_BASE</code>	<code>0x002000</code>	Base of <code>ENABLE[ctx][word]</code> registers
<code>ENABLE_STRIDE</code>	<code>0x000080</code>	Byte stride between contexts in ENABLE bank
<code>CONTEXT_BASE</code>	<code>0x200000</code>	Base of THRESHOLD / CLAIM-COMPLETE blocks
<code>CONTEXT_STRIDE</code>	<code>0x001000</code>	Byte stride between context control blocks
<code>CONTEXT_THR_OFF</code>	<code>0x000000</code>	Offset of <code>THRESHOLD</code> within a control block
<code>CONTEXT_CC_OFF</code>	<code>0x000004</code>	Offset of <code>CLAIM/COMPLETE</code> within a block
<code>WINDOW_SIZE</code>	<code>0x400000</code>	Total decoded window (4 MB)

5. Block diagram & port list

5.1 Block diagram

SMC PLIC — Top-Level Block Diagram



5.2 Ports

Port	Direction	Type	Description
<code>reg_socket</code>	target	<code>simple_target_socket</code>	TLM-2.0 register-access socket (AXI4-Lite-style)
<code>src_in[i]</code>	input	<code>sc_in<bool></code> × N	Source <code>i+1</code> (active-high, level-sensitive)
<code>ctx_out[c]</code>	output	<code>sc_out<bool></code> × C	Output for context <code>c</code> (active-high)
<code>rst_n_i</code>	input	<code>sc_in<bool></code>	Active-low synchronous reset

$N = num_sources$, $C = num_contexts$.

5.3 Context mapping (default `num_contexts = 8`)

ctx	RDL name	Hart	Privilege	Drives
0	CORE0_MEIP	0	M	MEIP[0]
1	CORE1_MEIP	1	M	MEIP[1]
2	CORE2_MEIP	2	M	MEIP[2]
3	CORE3_MEIP	3	M	MEIP[3]
4	CORE0_SEIP	0	S	SEIP[0]
5	CORE1_SEIP	1	S	SEIP[1]
6	CORE2_SEIP	2	S	SEIP[2]
7	CORE3_SEIP	3	S	SEIP[3]

The model groups all M-mode contexts first, then all S-mode contexts, to match the RDL ordering and the firmware driver.

6. Theory of operation

This section describes how the PLIC operates from a conceptual viewpoint, independently of the register-level details given in §7. It walks through the major architectural blocks, the lifecycle of a single interrupt event, and the steady-state interactions between hardware and software.

6.1 Operating principle

The PLIC sits between the device-level interrupt sources of the SMC and the per-hart `MEIP` / `SEIP` lines exposed by the Rocket cores. Its job is to:

1. **Aggregate** N independent active-high, level-sensitive source lines into a single internal pending-bit array.
2. **Filter** that array, per consumer (called a *context*), against per-context **enable** and **threshold** masks.
3. **Arbitrate** among the eligible sources of each context to pick a single highest-priority winner.
4. **Hand off** the winner to software via an atomic *claim* operation, then accept a *complete* notification when the ISR is done.

The model has no clock of its own. All state changes are event-driven — an output transition occurs only in response to a source toggle, a register write, a claim, a complete, or a reset. Between events the PLIC is silent; this matches both the loosely-timed SystemC abstraction and the underlying RTL, where the PLIC is a purely combinational arbiter sitting on top of a small set of latches.

6.2 Architectural concepts

Concept	Definition
Source	A device-level interrupt input; identified by a 1-based source ID (1..N).
Gateway	The per-source edge-detector that converts the source's active-high level into a pending message. Hidden behind <code>src_method</code> + <code>claim_in_flight</code> .
Pending bit	A latched flag indicating that a source has fired and has not yet been claimed.
Context	A <code>{hart, privilege-mode}</code> pair — i.e. one consumer of interrupts. SMC has 8 contexts (4 cores × {M, S}).
Enable bit	Per-source, per-context mask. Allows a context to opt in or out of a specific source.
Priority	Per-source 3-bit value (0..7). 0 disables the source globally; 7 is the highest.
Threshold	Per-context 3-bit cut-off. Only sources with <code>priority > threshold</code> are eligible.
Claim	Atomic CPU-side read of <code>CLAIM/COMPLETE</code> : returns the winning source ID and locks the source until complete.
Complete	CPU-side write of the source ID to <code>CLAIM/COMPLETE</code> : releases the in-flight lock and re-arms if the line is still high.
In-flight latch	Per-source flag asserted between claim and complete. Suppresses re-pend so the same source cannot fire while its ISR is running.

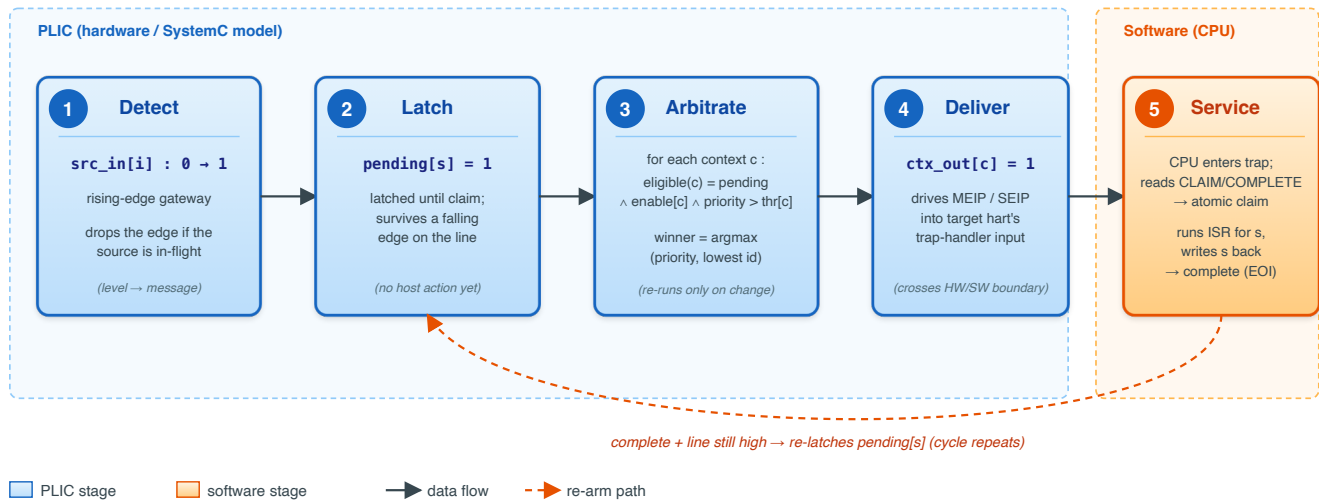
These eleven concepts are the entire mental model; everything else in this document is bookkeeping (register addressing, error responses, reset wiring) built on top of them.

6.3 Pipeline view

A single interrupt event flows through the PLIC in five stages:

Interrupt Lifecycle — 5-Stage Pipeline

Stages 1–4 are inside the PLIC. Stage 5 is software (ISR / driver).



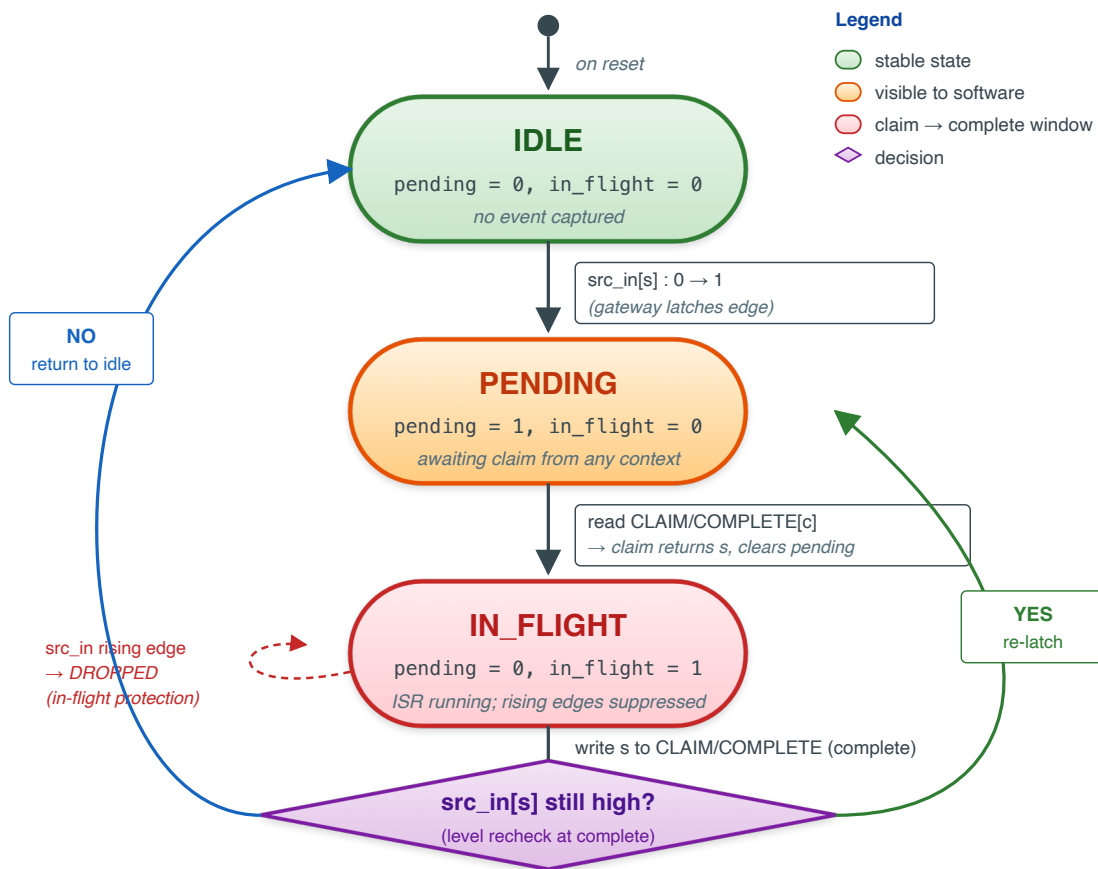
Stages 1–4 are entirely the PLIC's responsibility. Stage 5 happens in software (the ISR / driver in `riscv_plic0.c`); the PLIC only observes the resulting CLAIM/COMPLETE accesses.

6.4 Source-level lifecycle

A given source ID `s` moves through a small state machine. The PLIC keeps two bits of state per source — `pending[s]` and `in_flight[s]` — which together encode three reachable states:

Per-Source State Machine

State per source ID $s = (\text{pending}[s], \text{in_flight}[s])$



Two key properties fall out of this state machine:

- **Edge-trigger, level-recheck.** A rising edge is required to *enter* PENDING from IDLE, but the PLIC re-checks the *level* at complete time. A device that holds its line high will keep firing, one ISR at a time, until the device clears the cause. A device that pulses briefly will still be serviced — the rising edge is captured even if the level is gone before the ISR reads it.
- **One in-flight at a time per source.** Because rising edges are ignored while in IN_FLIGHT, the PLIC never delivers the same source twice concurrently to two different contexts. This is the **gateway in-flight protection** mandated by RISC-V PLIC v1.0 §4.

6.5 Per-context arbitration

Each context independently decides which source it wants to see. Three pieces of per-context state are involved:

- `enable[ctx][s]` — opt-in mask, one bit per source.

- `threshold[ctx]` — minimum priority (exclusive) the context is willing to handle.
- `claim_in_flight[s]` — global to all contexts (not per-context).

For context `c` the **eligible set** is computed on the fly from the shared `pending[]` array:

```
eligible(c) = { s : pending[s] = 1
                ^ enable[c][s] = 1
                ^ priority[s] > 0
                ^ priority[s] > threshold[c] }
```

`ctx_out[c]` is asserted iff `eligible(c) ≠ ∅`. A claim from context `c` returns the unique winner:

```
winner(c) = argmax over s ∈ eligible(c) by (priority[s], -s)
```

Highest priority wins; ties are broken by **lowest source ID**. Source 0 is never a winner — it is reserved to mean "no interrupt" and the arbitration loop iterates from 1 upwards.

Multi-context delivery

A single rising edge on source `s` can simultaneously raise `ctx_out[c]` for **every** context `c` that has `s` enabled and whose threshold permits it. Conversely, a claim from any single context removes `s` from arbitration in **every** context (because `pending[s]` becomes 0). This means the first context to claim wins, and other contexts whose `ctx_out` was raised by `s` will simply observe `s` disappear from their arbitration view. There is no "lost interrupt" — the source moves to IN_FLIGHT for the winning context's ISR to handle.

Cross-context complete

The RISC-V PLIC spec defines complete as a global EOI. The in-flight latch is per-source, not per-context. Therefore:

- Context A may legitimately claim a source and context B may legitimately complete it (e.g. in a hand-off between supervisor and machine modes).
- The PLIC does not check that the completing context matches the claiming context. Any context's write to `CLAIM/COMPLETE` with a valid source ID releases the in-flight lock.
- Spurious completes (out-of-range source IDs, or for sources not currently in flight) are silently dropped.

6.6 Threshold semantics — worked example

Suppose context `c` has `threshold[c] = 4`, and three sources are pending and enabled in `c`:

Source	Priority	Eligible for <code>c</code> ?	Reason
7	5	Yes	<code>5 > 4</code>
11	4	No	<code>4 > 4</code> (strict greater)
13	7	Yes	<code>7 > 4</code>

The arbiter picks source 13 (priority 7). If context `c` raises its threshold to 7, the eligible set becomes empty and `ctx_out[c]` falls even though sources 7 and 13 are still pending. They remain pending until claimed by some other context whose threshold allows them, or until the threshold is lowered again.

Setting `priority[s] = 0` is the global equivalent: source `s` becomes ineligible for **every** context, regardless of enable bits or thresholds.

6.7 Hardware ↔ software interaction

The PLIC's bus interface exposes exactly four classes of operation to software:

Operation	Effect
Configure (write priority/enable/thr)	Reshape the eligible set; may immediately raise/lower <code>ctx_out</code> .
Inspect (read priority/enable/thr/pending)	Pure read; no side effects.
Claim (read CLAIM/COMPLETE)	Atomic; mutates <code>pending[]</code> and <code>in_flight[]</code> .
Complete (write CLAIM/COMPLETE)	Mutates <code>in_flight[]</code> ; conditionally re-latches <code>pending[]</code> .

Every other access is either reserved (RAZ/WI) or an error response. In particular:

- There is no "force pending" register — the PLIC only believes its source-line inputs.
- There is no "clear pending" register — pending is cleared only by a successful claim.
- There is no shared global mask — masking is per-context (enable + threshold).

Boot sequence (simplified)

1. **Reset clears everything.** All priorities, enables, thresholds, pending bits, and in-flight latches are zero. `ctx_out[*]` is low. No source can fire (all priorities are 0, all enables are 0).
2. **Firmware programs priorities.** For each device source `s` that the system cares about, firmware sets `priority[s]` to a non-zero value. The source still cannot fire because all enables are zero.
3. **Firmware programs enables.** Per-context enable bits are set for the sources each context wants to receive.
4. **Firmware programs thresholds.** Typically `threshold[c] = 0`, meaning "deliver everything I'm enabled for".
5. **Firmware enables external interrupts** in the per-hart MIE/SIE CSR and sets MSTATUS.MIE / SSTATUS.SIE.
6. **Steady state.** The next rising edge of any enabled, non-zero- priority source raises the corresponding `ctx_out[c]` lines and the target hart traps into its external-interrupt handler.

ISR pattern

```
external_irq_handler(ctx):
    while true:
        s = read CLAIM/COMPLETE[ctx]    # atomic claim
        if s == 0: break                # no more pending
        device_handler[s]()             # service the device
        write s to CLAIM/COMPLETE[ctx]  # complete (EOI)
```

The loop drains every pending source for the context before returning, which is the standard PLIC programming model. If a device re-asserts its line during its own ISR (or a higher-priority device fires while the ISR runs), the next iteration will pick that up — claim returns the next eligible winner without needing to re-enter the trap.

6.8 Edge cases

Scenario	PLIC response
Claim with no pending source	Returns 0; no state change.
Claim while source line is still high	Pending cleared; re-arms only on next <i>complete</i> (level recheck). Rising edge will not re-pend until in-flight is cleared.
Complete without a prior claim	<code>in_flight[s]</code> was already 0 → no-op; if line is high, pending re-latches anyway.
Complete with src ID 0 or > num_sources	Silently dropped (per RISC-V PLIC spec).
Two contexts simultaneously raised by the same source	Source moves to IN_FLIGHT after the first claim; the second claim returns the next-best source (or 0).
Source raised before its enable bit is set	Pending bit latches anyway. Becomes deliverable as soon as enable + priority + threshold allow.
Firmware writes <code>priority[s] = 0</code> while <code>s</code> is pending	Pending bit is preserved, but <code>s</code> is now ineligible everywhere; <code>ctx_out</code> falls. Restoring a non-zero priority makes it deliverable again with the original pending bit intact.
Firmware writes <code>priority[s] = 0</code> while <code>s</code> is in flight	Complete still works; the source can re-pend after complete iff priority is non-zero again at that moment and the line is high.
Reset asserted mid-claim	All state is cleared; any in-flight ISR that completes after reset will write to <code>CLAIM/COMPLETE</code> with a stale source ID, which is silently dropped.

6.9 Steady-state characteristics

Quantity	Value (default config: 336 src, 8 ctx)
Maximum sustained interrupt rate	Bounded by claim/complete latency (model: 2 ns each)
Worst-case arbitration time	Linear in <code>num_sources</code> per context (~336 comparisons)
Worst-case output recompute	<code>num_sources × num_contexts</code> ≈ 2,688 comparisons
Internal state size	~1.9 KB (see <code>02_PLIC_LowLevel_Design.md §15</code>)
Output transitions per source event	≤ <code>num_contexts</code> (one per affected context)
Ordering between source events	Preserved (event-driven, single-threaded scheduler)

In firmware terms: the PLIC's contribution to ISR latency is one `b_transport` access per claim and one per complete. Everything else — trap delivery, register save/restore, device-handler runtime — is outside the PLIC's responsibility and outside this document's scope.

7. Register map

All offsets are **byte offsets relative to the PLIC base address**. Accesses must be 32-bit, naturally aligned. The full window size is 4 MB.

7.1 Region summary

Range	Region	Access	Size
<code>0x000000 .. 0x00FFC</code>	<code>PRIORITY[0..1023]</code>	RW	4 B × (N+1) used
<code>0x001000 .. 0x001FFC</code>	<code>PENDING[0..31]</code>	RO	4 B × [(N+1)/32] used
<code>0x002000 .. 0x1FFFFC</code>	<code>ENABLE[ctx][word]</code>	RW	4 B per word, 0x80/ctx
<code>0x200000 .. 0x3FFFFC</code>	<code>THRESHOLD[ctx] + CLAIM/COMPLETE[ctx]</code>	RW	0x1000 per ctx
≥ <code>0x400000</code>	Out-of-window	—	<code>TLM_ADDRESS_ERROR</code>

7.2 PRIORITY — `0x000000 + 4·src`

Bits	Field	Access	Reset	Description
31:3	reserved	RAZ/WI	0	Always reads 0; writes are dropped.
2:0	priority	RW	0	0 = disabled, 1 = lowest, 7 = highest.

- `PRIORITY[0]` is **reserved** (RAZ/WI). Source ID 0 is never delivered.
- For `src > num_sources` the address is in-range but reads as 0 and writes are ignored.

7.3 PENDING — `0x001000 + 4·word`

Bits	Field	Access	Reset	Description
31:0	pending[word*32+j]	RO	0	Bit <code>j</code> reflects pending state of source <code>word*32 + j</code> .

- Software writes are **silently discarded** (the field is software-RO).
- Bit 0 of word 0 (source 0) is permanently 0.
- For 336 sources, only words 0..10 are populated; bits beyond `num_sources` are always 0.

7.4 ENABLE — `0x002000 + 0x80·ctx + 4·word`

Bits	Field	Access	Reset	Description
31:0	enable[ctx][word*32+j]	RW	0	Bit <code>j</code> enables source <code>word*32 + j</code> for context <code>ctx</code> .

- Bit 0 of word 0 (`enable[ctx][0]`) is **forced to 0** on every write (source 0 is reserved).
- Bits beyond `num_sources` in the last populated word are masked to 0.
- Each context has 11 enable words for the default 336 sources; the unused words 11..31 within a context's 0x80-byte block are RAZ/WI.

7.5 THRESHOLD — `0x200000 + 0x1000·ctx + 0x000`

Bits	Field	Access	Reset	Description
31:3	reserved	RAZ/WI	0	Always reads 0; writes are dropped.
2:0	threshold	RW	0	A source is delivered to context <code>ctx</code> iff <code>priority[src] > threshold[ctx]</code> .

7.6 CLAIM/COMPLETE — `0x200000 + 0x1000·ctx + 0x004`

Op	Behaviour
Read	Atomically claim the highest-priority pending source for context <code>ctx</code> . Returns the source ID (0 if none); clears <code>pending[src]</code> ; sets internal <code>claim_in_flight[src]</code> .
Write	Complete the source whose ID is written. Clears <code>claim_in_flight[src]</code> ; if <code>src_in[src]</code> is still high, re-latches <code>pending[src]</code> .

- `transport_dbg` reads of this register **do not claim** — they return `best_pending(ctx)` without side effects.

- Writes of out-of-range source IDs are silently dropped (per RISC-V PLIC spec: the PLIC must tolerate spurious EOI writes).
- `complete()` is **context-agnostic** — the RISC-V PLIC spec defines complete as global, not per-context. Cross-context completion is legal.

7.7 Address arithmetic helpers

```
prio_addr(src)           = 0x000000 + 4 * src
pending_addr(word)       = 0x001000 + 4 * word
enable_addr(ctx, word)   = 0x002000 + 0x80 * ctx + 4 * word
threshold_addr(ctx)      = 0x200000 + 0x1000 * ctx + 0x0
claim_complete_addr(ctx) = 0x200000 + 0x1000 * ctx + 0x4
```

8. Functional behaviour

8.1 Source latching (gateway)

The PLIC sees device sources as **active-high, level-sensitive**. The internal gateway converts the level to an interrupt-pending message:

- Compare current `src_in[i]` against the previous sampled level.
- On a **rising edge** (`0 → 1`) of `src_in[i]`:
 - If `claim_in_flight[i+1]` is **clear**, set `pending[i+1]`.
 - If it is **set**, drop the edge (the source is in service).
- A **falling edge** (`1 → 0`) **does not clear** `pending`. Pending is level-latched until claimed.
- On `complete(ctx, src)`, if `src_in` is still high and `pending[src]` is clear, re-latch `pending[src]` immediately.

This guarantees:

- A device whose line stays asserted does **not** fire continuously during a single ISR.
- A device that pulses its line briefly is **not** missed; the rising-edge latch captures it even if the level is gone before claim.

8.2 Per-context arbitration

For each context `c`, the PLIC computes the eligible source set:

```

eligible(c) = { src ∈ [1..N] : pending[src]
                ∧ priority[src] > 0
                ∧ priority[src] > threshold[c]
                ∧ enable[c][src] }

```

The output `ctx_out[c]` asserts iff `eligible(c)` is non-empty.

The **claim** for context `c` returns:

```

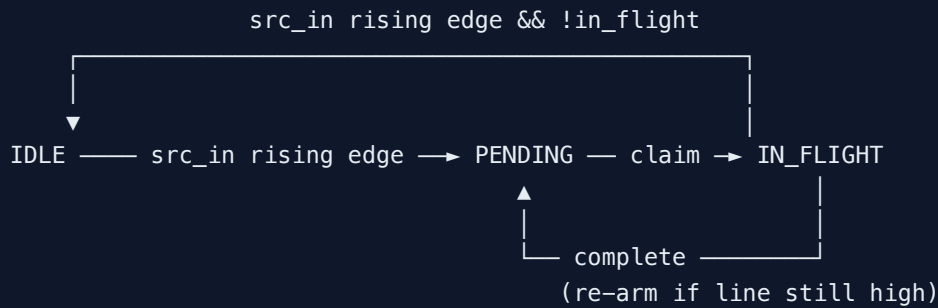
argmax over eligible(c) by (priority[src], -src_id)

```

Highest priority wins; ties broken by **lowest source ID**. If `eligible(c)` is empty, claim returns `0`.

8.3 Claim / complete handshake

State machine per source ID:



Operation	<code>pending</code>	<code>claim_in_flight</code>	<code>ctx_out</code> after
Source rising edge	$0 \rightarrow 1$	unchanged	recomputed
Source falling edge	unchanged	unchanged	recomputed
Claim (returns <code>src</code>)	$1 \rightarrow 0$	$0 \rightarrow 1$	recomputed
Claim (returns <code>0</code>)	unchanged	unchanged	unchanged
Complete, line low	unchanged	$1 \rightarrow 0$	recomputed
Complete, line high	$0 \rightarrow 1$	$1 \rightarrow 0$	recomputed
Spurious complete	unchanged	unchanged	recomputed

8.4 Source 0 reservation

The RISC-V PLIC spec reserves source ID 0 as "no interrupt". This is enforced in three places:

1. `PRIORITY[0]` is RAZ/WI.
2. Bit 0 of every `ENABLE[ctx][0]` is forced to 0 on writes.

3. Arbitration iterates from `src = 1` upwards; source 0 is never returned as a claim result.

8.5 Output-update model

Outputs are **edge-driven, not periodic**. `ctx_out[c]` transitions only in response to:

- A change of `src_in[*]`.
- A register write (priority / enable / threshold / claim / complete).
- A reset assertion or de-assertion.

There is no periodic refresh. From the firmware viewpoint the new `ctx_out` value is observable on the next register read after the triggering event.

9. Reset behaviour

The PLIC has a single **active-low, synchronous** reset input (`rst_n_i`), driven from the SMC reset unit's `rst_core_smc_n` domain (per `01_SMC_Architecture.md §3`).

On the falling edge of `rst_n_i`:

State	Reset value
<code>priority[*]</code>	0
<code>pending[*]</code>	0
<code>enable[*][*]</code>	0
<code>threshold[*]</code>	0
<code>claim_in_flight[*]</code>	0
<code>last_level[*]</code>	0
<code>ctx_out[*]</code>	0

The next rising edge of any `src_in[i]` after de-assertion of reset freshly latches a new pending bit (sources held high through reset are re-armed).

Reset has no effect on the AXI4-Lite contract — the bus interface remains responsive throughout the reset assertion (this is a transaction-level abstraction; in the LT model resets are zero-time).

10. Bus interface (TLM-2.0 / AXI4-Lite)

10.1 Socket type

```
tlm_utils::simple_target_socket<plic> reg_socket;
```

Registered callbacks:

```
reg_socket.register_b_transport  (this, &plic::b_transport);  
reg_socket.register_transport_dbg (this, &plic::transport_dbg);
```

DMI is **deliberately not registered**.

10.2 Transaction contract

Field	Required value	Notes
command	TLM_READ_COMMAND / TLM_WRITE_COMMAND	TLM_IGNORE_COMMAND returns TLM_COMMAND_ERROR_RESPONSE
address	4-byte aligned, < WINDOW_SIZE	else error (see §11)
data_length	4	else TLM_BURST_ERROR_RESPONSE
streaming_width	4	for compatibility
byte_enable_ptr	nullptr	byte enables ignored
dmi_allowed	(set to false on every txn)	DMI never granted

10.3 Timing annotation

`b_transport` adds `sc_time(access_delay_ns_p_.get_value(), SC_NS)` to the supplied delay reference. The delay value is read from the `access_delay_ns` CCI parameter (default **2.0 ns**, mutable at run-time). `transport_dbg` adds nothing (zero-time per TLM-2.0 convention).

The PLIC itself never initiates traffic; it has no quantum keeper and participates in whatever quantum the caller is running.

10.4 Optional `smc_axi_extension`

The shared SMC GP extension carrying `source_id`, `prot`, `cacheable`, `non_secure`, `axi_id`, `axi_user` is **honoured but not enforced** here. Access-control filtering belongs to the upstream `axi_filter` per `02_SMC_IP_LowLevel_Design.md §2`.

10.5 `transport_dbg`

Symmetric to `b_transport` for priority, enable, threshold, and pending registers, with one critical difference: **debug reads of `CLAIM/COMPLETE` do not claim**. They return the current `best_pending(ctx)` value with zero side effects. Debug writes are forwarded normally — including to `CLAIM/COMPLETE`, which still performs a `complete()` (this is intentional; it is a legitimate stimulus path for the test bench).

11. Error handling

Condition	Detection	Response
Address $\geq$ <code>WINDOW_SIZE</code>	<code>b_transport</code> precheck	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
Misaligned address	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
<code>data_length $\neq$ 4</code>	<code>b_transport</code> precheck	<code>TLM_BURST_ERROR_RESPONSE</code>
Unsupported command	<code>b_transport</code> switch	<code>TLM_COMMAND_ERROR_RESPONSE</code>
In-range, reserved register / hole	<code>reg_read</code> / <code>reg_write</code>	<code>TLM_OK_RESPONSE</code> (RAZ/WI)
Construction with <code>num_sources</code> out of range	Constructor	<code>SC_REPORT_FATAL</code> at elab time
Construction with <code>num_contexts < 1</code>	Constructor	<code>SC_REPORT_FATAL</code>
Out-of-range <code>complete(src)</code>	<code>complete()</code>	Silently dropped (per spec)

Runtime failures use TLM response codes so the surrounding test bench / firmware sees a deterministic error on the bus rather than a simulator abort. `SC_REPORT_FATAL` is reserved for elaboration-time misconfiguration.

12. Programming model

12.1 Initialisation sequence (firmware)

1. Optionally set `THRESHOLD[ctx] = 0` for every context the hart owns.
2. For each source `s` to be enabled:
 - Write a non-zero `PRIORITY[s]`.
 - Set the corresponding bit in `ENABLE[ctx][s/32]`.
3. Enable external interrupts in the per-hart MIE/SIE CSR.
4. Set the per-hart `MEIE` / `SEIE` bit; set `MSTATUS.MIE` / `SSTATUS.SIE`.

The firmware driver `riscv_plic0.c` performs exactly this sequence and must run **unmodified** against the SMC PLIC model.

12.2 ISR template

```
void plic_isr(unsigned ctx) {
    while (1) {
        uint32_t src = readl(PLIC_BASE + claim_complete_addr(ctx));
        if (src == 0) break;          /* no more pending */
        dispatch_handler(src);        /* run device-specific handler */
        writel(src, PLIC_BASE + claim_complete_addr(ctx)); /* complete */
    }
}
```

12.3 Worked example

Goal: deliver source 7 (priority 5) to context 0 with threshold 4.

```
write32(prio_addr(7),      5);          // priority = 5
write32(enable_addr(0, 0), 1u << 7);    // enable bit 7 in ctx 0, word 0
write32(thr_addr(0),      4);          // threshold = 4 (5 > 4)
//
src_in[6] = 1;                        // device asserts source 7
// (src_in[i] → source i+1)

// pending[7] = 1 ; ctx_out[0] = 1
//
src = read32(cc_addr(0));              // returns 7; pending[7] = 0
// ... ISR body ...
write32(cc_addr(0), src);              // complete; if line still
// high, pending[7] = 1
// and ctx_out[0] = 1 again
```

13. Compliance matrix

Requirement	Specified by	Compliance
3-bit priority and threshold	<code>plic.rdl</code>	✓ enforced via mask on write
Priority 0 = source disabled	RISC-V PLIC §3	✓ filtered in <code>best_pending()</code>
Strict <code>priority > threshold</code>	RISC-V PLIC §3	✓ filtered in <code>best_pending()</code>
Lowest-source-ID tie-break	RISC-V PLIC §3	✓ ascending scan in <code>best_pending()</code>
Source 0 reserved	RISC-V PLIC §3	✓ RAZ/WI on PRIORITY/ENABLE; never returned by claim
Pending latched on rising edge, persists across falling edges	RISC-V PLIC §4	✓ in <code>src_method</code> and <code>complete()</code>
<code>claim_in_flight</code> blocks re-pend until complete	RISC-V PLIC §4	✓ guard in <code>src_method</code>
Re-arm on complete if line still high	RISC-V PLIC §4	✓ in <code>complete()</code>
Spurious complete tolerated	RISC-V PLIC §4	✓ silent drop
Cross-context complete legal	RISC-V PLIC §4	✓ <code>complete()</code> is <code>ctx</code> -agnostic
32-bit aligned register access only	<code>plic.rdl</code> SW= <code>r/rw</code> 4 B fields	✓ enforced; else <code>TLM_BURST_ERROR_RESPONSE</code>
4 MB window	RDL <code>addrmap</code> size	✓ enforced via <code>WINDOW_SIZE</code>
<code>riscv_plic0.c</code> runs unmodified	SMC FW contract	✓ verified by integration test plan

13.1 Documented deviations

None. All behaviour matches the RISC-V PLIC v1.0 spec, the RDL, and the firmware driver.

13.2 Unmodelled features

Item	Reason
Approximately-Timed (AT) protocol	LT model is sufficient for SW bring-up.
Per-source synchroniser latency	Folded into the LT zero-time abstraction.
Edge-only source mode	All SMC sources are level; not required.
DMI fast path	Claim has read side effects (incompatible with DMI).

14. Revision history

Version	Date	Author	Notes
1.0	2025-04-26	SMC modelling	Initial release — matches <code>plic.rdl</code> and <code>riscv_plic0.c</code> .
1.1	2026-05-12	SMC modelling	§4 rewritten: <code>num_sources</code> , <code>num_contexts</code> , and <code>access_delay_ns</code> are now OSCI CCI parameters (<code>cci_param</code>). <code>plic_cfg</code> constructor argument retained for backward-compatible default injection. §10.3 updated to reflect <code>access_delay_ns_p</code> CCI param.

15. Glossary

- **PLIC** — Platform-Level Interrupt Controller (RISC-V).
- **Context** — A `{hart, privilege}` pair. SMC has 8 contexts: `{core 0..3} × {M-mode, S-mode}`.
- **Source** — A device-level interrupt input (1.. `num_sources`).
- **Claim** — Atomic CPU-side read of `CLAIM/COMPLETE` ; returns the highest-priority pending source ID and clears its pending bit.
- **Complete** — CPU-side write of source ID to `CLAIM/COMPLETE` ; signals end-of-handler so the source can re-pend.
- **In-flight** — A source between claim and complete. New rising edges on its line are ignored; if the line is still high at complete, pending is re-armed.
- **Threshold** — Per-context priority cut-off; sources with `priority ≤ threshold` are masked out of arbitration.
- **Gateway** — Per-source spec-level abstraction that converts the raw source level into IP/EOI messages. In this model the gateway is folded into `src_method` + `claim_in_flight_`.
- **LT (Loosely-Timed)** — TLM-2.0 modeling style: `b_transport` , optional DMI, coarse `sc_time` annotations.
- **AXI4-Lite** — Simplified ARM AMBA bus protocol with 32-bit aligned reads/writes, no bursting; the on-chip protocol the PLIC's TLM socket abstracts.
- **DMI** — Direct Memory Interface; lets initiators bypass `b_transport` for memory-like targets. Refused by the PLIC.
- **RDL** — SystemRDL register-description language; the authoritative PLIC register layout lives in `hw/smc/smc_cpu/data/registers/rdl/plic.rdl`.

End of document.